

Índice

Guía de Estudio — Tema 08: Paradigma OO — TypeScript + Smalltalk	1
0. Cómo usar esta guía	1
1. Objetivos de Aprendizaje	2
2. Conocimientos Previos	2
3. Desarrollo Teórico	3
3.1 Origen y filosofía del OO	3
3.2 Los 4 pilares del OO en detalle	4
3.3 Smalltalk en profundidad	7
3.4 TypeScript OO en profundidad	9
3.5 Dominio completo — Sistema de figuras geométricas	11
4. Puntos Clave del Tema	13
5. Autoevaluación	13
Respuestas de Autoevaluación	14
6. Glosario	16
7. Referencias	17
Bibliografía del curso (peer-reviewed)	17
Para profundizar en Smalltalk	17
Para profundizar en TypeScript	17
Entorno de práctica	17

Guía de Estudio — Tema 08: Paradigma OO — TypeScript + Smalltalk

Para el alumno: este documento es tu compañero de estudio autónomo. Cubre en profundidad todo lo visto en clase y te prepara para comprender y aplicar el paradigma OO.

Materia: Paradigmas y Lenguajes de Programación 2026

Docente: Matías Gel — UNTDF / IDEI

Duración de la clase: 120 min | **Módulo:** IV — Paradigma OO | Semana 7

Fecha: 2026-04-28

Fuentes: Gabbrielli & Martini (2023) Cap.10; Sebesta (2019) Cap.12; Sweller/Chen CLT (2023)

0. Cómo usar esta guía

Esta guía **expande** la clase, no la reemplaza. Estructura:

1. **Objetivos de aprendizaje** — qué vas a saber al terminar
2. **Conocimientos previos** — qué necesitás saber antes
3. **Desarrollo teórico** — conceptos en profundidad con ejemplos
4. **Ejemplos trabajados paso a paso**
5. **Puntos clave del tema**
6. **Autoevaluación** — 20 preguntas con respuesta al final

7. Glosario

8. Referencias

Tiempo estimado de lectura activa: 3–4 horas en 2 sesiones de 90 min.

Método recomendado: - Lee una sección completa. - Abrió un playground TypeScript (<https://www.typescriptlang.org/play>) y **tipeá cada ejemplo**. - Para Smalltalk, usá Pharo Playground en línea: <https://pharo.org/> - Modificá los ejemplos y observá qué cambia. - Antes de pasar de sección: explicá el concepto en voz alta con tus propias palabras.

1. Objetivos de Aprendizaje

Al terminar de estudiar este tema debés poder:

#	Objetivo	Nivel Bloom
1	Enumerar los 4 pilares del OO y definir cada uno	Recordar
2	Explicar qué significa “todo es un objeto” en Smalltalk y por qué TypeScript no lo cumple igual	Comprender
3	Implementar una jerarquía de clases con herencia y polimorfismo en TypeScript	Aplicar
4	Comparar el mismo dominio implementado en Smalltalk vs. TypeScript	Analizar
5	Distinguir tipado estructural (TypeScript) de tipado nominal (Java) y duck typing dinámico (Smalltalk)	Analizar
6	Argumentar qué ventajas y compromisos introduce TypeScript respecto a OO puro	Evaluar
7	Modelar un mini-dominio en TypeScript aplicando los principios OO vistos	Crear

2. Conocimientos Previos

Antes de este tema debés dominar:

Tema	Dónde se vio
TypeScript básico — tipos, funciones, arrow functions	T03-T05
Inmutabilidad y funciones puras (paradigma funcional)	T03-T05
Paradigma lógico — Prolog, declaratividad	T06-T07
Sistemas de tipos básicos	T01

Si alguno está flojo, revisá los apuntes del tema correspondiente antes de continuar.

3. Desarrollo Teórico

3.1 Origen y filosofía del OO

3.1.1 Simula 67 — el primer paso El paradigma orientado a objetos nació en **Simula 67** (Nygaard & Dahl, Noruega). Fue diseñado para simular sistemas del mundo real — un barco en una simulación tiene estado (posición, velocidad) y comportamiento (moverse, frenar). Múltiples barcos son instancias de la misma clase.

Simula no era “OO puro” — era ALGOL extendido con clases. Pero introdujo la idea clave: **agrupar datos con las operaciones que los modifican** (encapsulamiento).

Antes de Simula, el paradigma dominante era el imperativo: los datos eran estructuras (structs, records) completamente separadas de las funciones que los procesaban. No había ningún mecanismo que dijera “estos datos pertenecen a estas funciones”.

3.1.2 Alan Kay y la visión de Smalltalk Alan Kay (Xerox PARC, 1970s) acuñó el término “*Object-Oriented Programming*” y creó Smalltalk. Su visión era radical:

“The big idea is ‘messaging’ — [...] The key in making great and growable systems is much more to design how its modules communicate rather than what their internal properties and behaviors should be.”

Para Kay, OO **no era sobre clases** — era sobre **mensajes entre objetos autónomos**. Cada objeto es una especie de pequeña computadora que recibe mensajes y responde con un resultado.

Los tres principios originales de Smalltalk: 1. **Todo es un objeto** — sin excepciones: números, booleanos, clases, métodos 2. **Los objetos se comunican únicamente por mensajes** 3. **Cada objeto tiene su propia memoria** — encapsulamiento absoluto

3.1.3 Evolución — de la pureza al pragmatismo La historia del OO es la historia de hacer concesiones al mundo real del desarrollo:

Lenguaje	Pureza OO	Concesión principal
Smalltalk-80	Máxima	Sin tipado estático, sin primitivos
Java	Alta	Sin metaclasses, primitivos (int, boolean)

Lenguaje	Pureza OO	Concesión principal
Python	Media	Duck typing dinámico, herencia múltiple compleja
TypeScript	Media	Primitivos, sin metaclasses, tipado estructural

Cada generación ganó en herramientas y productividad de equipo, perdió en pureza filosófica.

3.2 Los 4 pilares del OO en detalle

3.2.1 Encapsulamiento Definición: el estado de un objeto solo es accesible a través de sus métodos públicos. Los detalles internos están ocultos.

En Smalltalk: las variables de instancia son **siempre privadas**. No hay acceso directo desde fuera del objeto — la única forma es a través de mensajes (*accessors*).

```
Animal >> nombre
  ^ nombre                "getter – mensaje nombre"

Animal >> nombre: unNombre
  nombre := unNombre      "setter – mensaje nombre:"
```

En TypeScript: controlado con modificadores de acceso:

```
class CuentaBancaria {
  private saldo: number;    // solo accesible internamente
  readonly titular: string; // immutable post-construcción

  constructor(titular: string, saldoInicial: number) {
    this.titular = titular;
    this.saldo = saldoInicial;
  }

  depositar(monto: number): void {
    if (monto > 0) this.saldo += monto;
  }

  retirar(monto: number): boolean {
    if (monto > 0 && monto <= this.saldo) {
      this.saldo -= monto;
      return true;
    }
    return false;
  }

  obtenerSaldo(): number {
    return this.saldo; // solo se puede leer, no modificar directamente
  }
}
```

```

    }
}

const cuenta = new CuentaBancaria("Ana", 1000);
cuenta.depositar(500);
console.log(cuenta.obtenerSaldo()); // 1500
// cuenta.saldo = 9999;           // ❌ Error de compilación

```

Por qué importa: el encapsulamiento permite cambiar la implementación interna sin romper el código que usa la clase. Si mañana cambiamos cómo se almacena saldo, ningún cliente de CuentaBancaria se rompe.

3.2.2 Abstracción Definición: exponer solo lo necesario para usar un objeto, ocultar la complejidad de cómo funciona.

Ejemplo: cuando usás `.sort()` en un array TypeScript, no necesitás saber qué algoritmo usa internamente (quicksort, timsort, depende del motor). La interfaz es simple: `array.sort()`.

Clases abstractas en TypeScript:

```

abstract class Forma {
    protected color: string;
    constructor(color: string) { this.color = color; }

    // Método abstracto — obliga a las subclases a implementarlo
    abstract area(): number;
    abstract perimetro(): number;

    // Método concreto — usa los abstractos sin saber su implementación
    esMasGrande(otra: Forma): boolean {
        return this.area() > otra.area();
    }

    describirse(): string {
        return `${this.constructor.name} (color=${this.color}): area=${this.area().toFixed(
    }
}

```

En Smalltalk: no hay palabras clave `abstract` — la convención es usar `^ self subclassResponsibility` que lanza un error en runtime si la subclase no redefine el método.

3.2.3 Herencia Definición: una clase (subclase) puede reusar y especializar el comportamiento de otra clase (superclase).

Regla de oro — “es-un”: usá herencia solo cuando la subclase **es-un** tipo de la superclase. Un Perro es-un Animal. Un Administrador es-un Usuario.

Error común — herencia por reutilización: usar herencia solo para reusar código aunque no haya relación semántica. Esto viola el principio de Liskov (LSP).

```
// Correcto — Perro ES UN Animal
class Animal { hablar(): string { return "..."; } }
class Perro extends Animal { hablar(): string { return "Guau!"; } }

// Incorrecto — Stack NO ES UN Array
// class Stack extends Array<number> { ... }
// Mejor: Stack TIENE UN Array (composición)
class Stack<T> {
  private items: T[] = [];
  push(item: T): void { this.items.push(item); }
  pop(): T | undefined { return this.items.pop(); }
  peek(): T | undefined { return this.items[this.items.length - 1]; }
}
```

En Smalltalk:

```
Animal subclass: #Perro
  instanceVariableNames: 'raza'
  category: 'Ejemplo'.

Perro >> hablar
  ^ 'Guau! Soy un ', raza
```

Herencia simple en ambos lenguajes: tanto Smalltalk como TypeScript permiten solo un padre directo (herencia simple). Java también. C++ permite herencia múltiple, pero genera problemas (problema del diamante).

3.2.4 Polimorfismo Definición: diferentes objetos pueden responder al mismo mensaje/llamada con comportamientos distintos.

Polimorfismo por herencia (override):

```
abstract class Forma {
  abstract area(): number;
}

class Circulo extends Forma {
  constructor(private radio: number) { super(); }
  area(): number { return Math.PI * this.radio ** 2; }
}

class Rectangulo extends Forma {
  constructor(private w: number, private h: number) { super(); }
  area(): number { return this.w * this.h; }
}

// El cliente no sabe (ni necesita saber) el tipo concreto
```

```
function imprimirArea(f: Forma): void {
  console.log(`Área: ${f.area().toFixed(2)}`);
}

imprimirArea(new Circulo(5));           // "Área: 78.54"
imprimirArea(new Rectangulo(4, 6));      // "Área: 24.00"
```

Polimorfismo por interfaz (sin herencia):

```
interface Ordenable {
  comparar(otro: Ordenable): number; // -1, 0, 1
}

class Temperatura implements Ordenable {
  constructor(private grados: number) {}
  comparar(otro: Temperatura): number {
    return Math.sign(this.grados - otro.grados);
  }
}

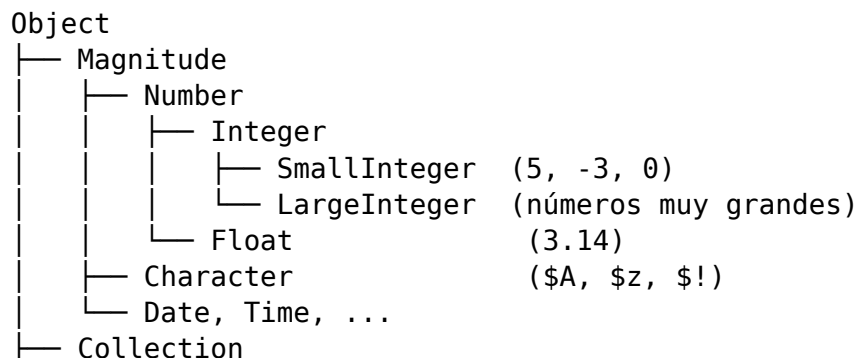
class Precio implements Ordenable {
  constructor(private valor: number) {}
  comparar(otro: Precio): number {
    return Math.sign(this.valor - otro.valor);
  }
}
```

En Smalltalk: el polimorfismo es siempre por despacho dinámico de mensajes. Si el objeto responde al mensaje, funciona — sin necesitar declarar implements:

```
"Funciona con cualquier objeto que entienda 'area'"
formas do: [:f | Transcript showCr: f area printString].
```

3.3 Smalltalk en profundidad

3.3.1 La jerarquía completa de objetos



		Array	(#(1 2 3))
		OrderedCollection	(lista dinámica)
		Set	(sin duplicados)
		Dictionary	(pares clave-valor)
		Boolean	
			True (true)
			False (false)
		UndefinedObject	(nil)

Todo en esta jerarquía hereda de Object. Esto significa que **cualquier objeto** puede responder a mensajes como: - printString — representación textual - class — retorna la clase del objeto - respondsTo: aSymbol — pregunta si el objeto entiende un mensaje - isNil — pregunta si es nil

3.3.2 La precedencia de mensajes En Smalltalk, los mensajes tienen precedencia: 1. **Unarios** — mayor precedencia: 5 factorial negated = -(5!) = -120 2. **Binarios** — media: $3 + 4 * 2 = (3+4)*2 = 14$ ← ¡no es 11! 3. **Palabra clave** — menor precedencia: a at: 1 put: b factorial

Los paréntesis cambian la precedencia:

```
3 + (4 * 2).    "→ 11 (con paréntesis)"
3 + 4 * 2.     "→ 14 (sin paréntesis – izquierda a derecha)"
```

Nota: en Smalltalk NO existen las reglas de precedencia matemática convencionales para los operadores binarios — todos tienen la misma precedencia y se evalúan de izquierda a derecha. Los paréntesis son obligatorios cuando querés + antes que *.

3.3.3 Bloques — lambdas de Smalltalk Los bloques en Smalltalk son objetos que representan código diferido (equivalentes a lambdas/arrow functions en TypeScript):

"Bloque sin argumento"

```
| b |
b := [Transcript show: 'hola'].
b value.    "→ imprime 'hola'"
```

"Bloque con un argumento"

```
| cuadrado |
cuadrado := [:x | x * x].
cuadrado value: 5.    "→ 25"
```

"Bloques como condicionales"

```
(3 > 2) ifTrue: [Transcript show: 'mayor'].
```

"Bloques como iteradores"

```
 #(1 2 3 4 5) do: [:n | Transcript showCr: n printString].
```

Comparación con TypeScript:


```
const cuadrado = (x: number) => x * x;
cuadrado(5); // 25

[1, 2, 3, 4, 5].forEach(n => console.log(n));
```

3.4 TypeScript OO en profundidad

3.4.1 Tipado estructural — la característica central TypeScript usa **tipado estructural** (también llamado duck typing estático). Dos tipos son compatibles si tienen la misma **forma** (estructura), sin importar si uno declara explícitamente implements al otro.

```
interface Punto2D {
  x: number;
  y: number;
}

// Clase explícita con implements
class PuntoA implements Punto2D {
  constructor(public x: number, public y: number) {}
}

// Literal de objeto — sin clase, sin implements
const puntoLiteral = { x: 3, y: 4 };

// Función que acepta cualquier Punto2D
function distanciaAlOrigen(p: Punto2D): number {
  return Math.sqrt(p.x ** 2 + p.y ** 2);
}

distanciaAlOrigen(new PuntoA(3, 4)); // 5
distanciaAlOrigen(puntoLiteral);     // 5 — estructuralmente compatible
distanciaAlOrigen({ x: 6, y: 8 });   // 5 — literal inline
```

Contraste con Java (tipado nominal):

```
// En Java, esto NO compilaría — puntoLiteral no declara implements Punto2D
Punto2D p = new HashMap<>(); // Error — HashMap no implementa Punto2D
```

Contraste con Smalltalk (duck typing dinámico):

```
"En Smalltalk, cualquier objeto que entienda 'x' y 'y' funciona — sin verificación previa
distanciaAlOrigen: p
  ^ ((p x raisedTo: 2) + (p y raisedTo: 2)) sqrt
```

3.4.2 extends vs. implements — cuándo usar cada uno

Herramienta	Cuándo usarla
extends (una sola clase)	Cuando la subclase es-un tipo de la superclase y quiere reusar implementación
implements (varias interfaces)	Cuando la clase necesita cumplir un contrato sin necesitar la implementación del padre
Composición + interfaz	Cuando querés reusar comportamiento sin relación semántica

Ejemplo de composición (preferible a herencia):

```
interface Logger {
  log(mensaje: string): void;
}

class ConsoleLogger implements Logger {
  log(mensaje: string): void {
    console.log(`[${new Date().toISOString()}] ${mensaje}`);
  }
}

// En vez de heredar de Logger, se inyecta
class Servicio {
  constructor(private logger: Logger) {}

  ejecutarTarea(nombre: string): void {
    this.logger.log(`Iniciando: ${nombre}`);
    // ... lógica
    this.logger.log(`Completado: ${nombre}`);
  }
}

const servicio = new Servicio(new ConsoleLogger());
servicio.ejecutarTarea("procesamiento");
```

3.4.3 Genéricos — polimorfismo paramétrico TypeScript permite definir clases y funciones que funcionan con cualquier tipo:

```
class Pila<T> {
  private elementos: T[] = [];

  push(item: T): void {
    this.elementos.push(item);
  }

  pop(): T | undefined {
```

```

    return this.elementos.pop();
}

peek(): T | undefined {
    return this.elementos[this.elementos.length - 1];
}

estaVacia(): boolean {
    return this.elementos.length === 0;
}
}

const pilaNumeros = new Pila<number>();
pilaNumeros.push(1);
pilaNumeros.push(2);
console.log(pilaNumeros.pop()); // 2

const pilaTextos = new Pila<string>();
pilaTextos.push("hola");
pilaTextos.push("mundo");
console.log(pilaTextos.peak()); // "mundo"

```

Restricciones en genéricos:

```

interface TieneNombre {
    nombre: string;
}

function saludar<T extends TieneNombre>(elemento: T): string {
    return `Hola, ${elemento.nombre}`;
}

saludar({ nombre: "Ana", edad: 25 }); // ❌ - tiene 'nombre'
// saludar({ edad: 25 });             // ❌ - falta 'nombre'

```

3.5 Dominio completo — Sistema de figuras geométricas

Este es el ejemplo central que usamos en clase. Te recomendamos implementarlo desde cero:

```

abstract class Figura {
    constructor(protected color: string) {}
    abstract area(): number;
    abstract perimetro(): number;

    describirse(): string {
        return `${this.constructor.name}(${this.color}) → área=${this.area().toFixed(2)}, p

```

```

    }

    esCongruente(otra: Figura): boolean {
        return Math.abs(this.area() - otra.area()) < 0.0001;
    }
}

class Circulo extends Figura {
    constructor(private radio: number, color: string) { super(color); }
    area(): number { return Math.PI * this.radio ** 2; }
    perimetro(): number { return 2 * Math.PI * this.radio; }
}

class Rectangulo extends Figura {
    constructor(private ancho: number, private alto: number, color: string) {
        super(color);
    }
    area(): number { return this.ancho * this.alto; }
    perimetro(): number { return 2 * (this.ancho + this.alto); }
}

class Triangulo extends Figura {
    constructor(
        private base: number,
        private lado1: number,
        private lado2: number,
        color: string
    ) { super(color); }
    area(): number {
        // Fórmula de Herón
        const s = (this.base + this.lado1 + this.lado2) / 2;
        return Math.sqrt(s * (s - this.base) * (s - this.lado1) * (s - this.lado2));
    }
    perimetro(): number { return this.base + this.lado1 + this.lado2; }
}

// Uso - polimorfismo
const figuras: Figura[] = [
    new Circulo(5, "rojo"),
    new Rectangulo(4, 6, "azul"),
    new Triangulo(3, 4, 5, "verde"),
];

figuras.forEach(f => console.log(f.describirse()));
// Circulo(rojo) → área=78.54, perímetro=31.42
// Rectangulo(azul) → área=24.00, perímetro=20.00
// Triangulo(verde) → área=6.00, perímetro=12.00

```

```
// Ordenar por área — tipado estructural en acción
figuras.sort((a, b) => a.area() - b.area());
console.log("\nOrdenadas por área:");
figuras.forEach(f => console.log(`  ${f.constructor.name}: ${f.area().toFixed(2)}`));
```

4. Puntos Clave del Tema

Estos son los conceptos centrales que debés dominar al terminar el tema:

1. **Los 4 pilares:** encapsulamiento, abstracción, herencia, polimorfismo — saber definirlos Y ejemplificarlos en código.
2. **OO puro vs. pragmático:**
 - Smalltalk → todo es objeto, mensajes, metaclasses, sin primitivos
 - TypeScript → primitivos, abstract, interfaces, duck typing estático
3. **Tipado:** nominal (Java) vs. estructural (TypeScript) vs. dinámico (Smalltalk)
4. **Cuándo usar extends vs. implements** — saber argumentar la decisión.
5. **abstract** → qué hace, cuándo usarlo, cómo se comporta en runtime vs. compilación.
6. **El dominio de figuras:** deberías poder implementarlo de memoria — es el ejercicio tipo de integración del tema.

5. Autoevaluación

Respondé estas preguntas. Las respuestas están al final de la sección.

Conceptual:

1. ¿Cuál es la diferencia entre `private` y `protected` en TypeScript?
2. ¿Qué significa que TypeScript usa tipado estructural? Dá un ejemplo donde eso hace diferencia.
3. ¿Por qué en Smalltalk no se pueden tener variables de instancia públicas?
4. ¿Qué pasa en Smalltalk si una subclase no redefine un método con `^ self subclassResponsibility`?
5. ¿Qué es una metaclass en Smalltalk?
6. ¿Cuál es la diferencia entre polimorfismo por herencia y por interfaz?
7. ¿Por qué la herencia de `Stack` `extends` `Array` es un anti-patrón?
8. ¿Qué es el Principio de Sustitución de Liskov (LSP)?

De código TypeScript:

9. ¿Qué produce este código? ¿Por qué?

```
abstract class A { abstract f(): number; g() { return this.f() * 2; } }
class B extends A { f() { return 3; } }
```

```
const b = new B();  
console.log(b.g());
```

10. ¿Compila este código? Si no, ¿por qué?

```
interface X { valor: number; }  
class C { valor = 5; }  
const obj: X = new C();
```

11. ¿Compila este código? Si no, ¿por qué?

```
abstract class Figura { abstract area(): number; }  
const f = new Figura();
```

12. Escribí una clase Cuadrado que extienda Rectangulo del dominio de figuras.
¿Tiene sentido semánticamente?

De Smalltalk:

13. ¿Qué diferencia hay entre =, == y := en Smalltalk?

14. ¿Qué hace este código?

```
| n |  
n := 5.  
n class.
```

15. ¿Cuáles son los tres tipos de mensajes en Smalltalk y cuál tiene mayor precedencia?

Integración de paradigmas:

16. ¿Cuándo es más conveniente resolver un problema con el paradigma funcional vs. el OO?

17. ¿En qué se parece el polimorfismo de Smalltalk al duck typing de Python?

18. ¿Qué diferencia hay entre instanciar new Animal() en TypeScript y Animal new en Smalltalk?

19. ¿Puedo usar una interfaz de TypeScript como tipo de retorno de una función? Dé un ejemplo.

20. ¿Qué sucede si una clase TypeScript implementa una interfaz pero no implementa todos sus métodos?

Respuestas de Autoevaluación

1. private → solo accesible dentro de la clase misma. protected → accesible en la clase y en todas sus subclases.
2. Dos tipos son compatibles si tienen la misma estructura (mismos campos/métodos), sin necesitar implements explícito. Ejemplo: { x: number, y: number } es compatible con interface Punto2D { x: number; y: number } sin declarar la interfaz.

3. En Smalltalk, el encapsulamiento es absoluto — las variables de instancia siempre son privadas. La única forma de acceder es con mensajes (accessors).
4. Se lanza un error en runtime: "This method is the responsibility of a subclass". No es un error en compilación — es una convención, no una restricción técnica del compilador.
5. Una metaclass es la clase de una clase. En Smalltalk, la clase `Animal` es un objeto — una instancia de su metaclass `Animal class`. Permite que las clases tengan sus propios métodos (equivalente a `static` en TypeScript).
6. **Por herencia:** una subclase redefine (override) un método de la superclase. **Por interfaz:** distintas clases no relacionadas implementan los mismos métodos declarados en una interfaz.
7. Viola el Principio de Liskov: un `Stack` NO puede usarse en todos los contextos donde se usa un `Array` — por ejemplo, no tiene sentido `stack[0] = 99` o `stack.splice(1, 2)`. La herencia aquí es por reutilización de código, no por relación semántica. Usar composición.
8. Si `S` es una subclase de `T`, entonces cualquier instancia de `S` debería poder usarse donde se use `T` sin alterar el comportamiento esperado del programa.
9. Produce 6. `g()` llama a `this.f()` → en `B`, `f()` devuelve 3 → $3 * 2 = 6$. Polimorfismo en acción — `g()` está en la clase abstracta pero usa el `f()` de la subclase.
10. Sí compila. TypeScript usa tipado estructural — `C` tiene valor: `number`, que es lo que pide `X`. No necesita `implements X`.
11. No compila. Error: Cannot create an instance of an abstract class. Las clases abstractas no se pueden instanciar directamente.
12. `Cuadrado` extends `Rectangulo` con `lado: number` (en vez de `ancho` y `alto` distintos). Semánticamente puede tener sentido (un cuadrado ES un rectángulo con `ancho === alto`), pero viola LSP: si alguien llama `cuadrado.ancho = 5` espera que el cuadrado cambie solo el ancho, pero para mantener la propiedad del cuadrado debería cambiar también `alto`. Esto es un ejemplo clásico de por qué la herencia debe usarse con cuidado.
13. `=` → igualdad de valor (contenido). `==` → identidad (mismo objeto en memoria). `===` → igualdad numérica (solo para números).
14. Asigna 5 a `n`. `n class` devuelve `SmallInteger` — la clase del objeto 5.
15. **Unarios** (mayor precedencia), **binarios** (media), **palabras clave** (menor). Dentro del mismo nivel, se evalúan de izquierda a derecha.
16. **Funcional** → transformaciones de datos sin estado mutable, procesamiento de colecciones, funciones puras, pipelines. **OO** → modelar entidades del mundo real con estado y comportamiento, sistemas con múltiples tipos relacionados, aplicaciones de larga vida con muchos objetos.
17. En ambos casos, la compatibilidad se basa en que el objeto “sepa responder” a ciertos métodos/mensajes — no en su tipo declarado. La diferencia es que Smalltalk lo verifica en runtime y Python también, mientras que TypeScript lo verifica en compilación (duck typing *estático*).
18. En TypeScript `new Animal()` invoca el método especial `constructor()`. En Smalltalk `Animal new` envía el mensaje `new` a la clase `Animal` — que es un objeto — y la clase ejecuta su método `new`. La diferencia es que en Smalltalk es un envío de mensaje ordinario, modificable por la metaclass.
19. Sí. Ejemplo: `function crearForma(): Forma { return new Circulo(5, "rojo"); }`. El tipo de retorno es la interfaz/clase abstracta, pero se retorna una instancia

concreta.

20. Error de compilación: Class 'X' incorrectly implements interface 'Y'.
Property 'z' is missing in type 'X' but required in type 'Y'.

6. Glosario

Término	Definición
Abstracción	Exponer solo la interfaz necesaria, ocultando la implementación interna
Clase abstracta	Clase que no puede instanciarse directamente — define un contrato para sus subclasses
Duck typing	“Si camina como un pato y grazna como un pato, es un pato” — compatibilidad por comportamiento
Encapsulamiento	Agrupar datos y comportamiento, ocultando los detalles internos
Herencia	Mecanismo por el cual una subclase reutiliza y especializa el comportamiento de su superclase
Interfaz	Contrato de métodos que una clase se compromete a implementar
Mensaje (Smalltalk)	La única forma de comunicación entre objetos — equivalente a llamada a método
Metaclass (Smalltalk)	La clase de una clase — permite que las clases tengan comportamiento propio
Método abstracto	Método declarado sin implementación — obliga a la subclase a implementarlo
Polimorfismo	Capacidad de diferentes objetos de responder al mismo mensaje con comportamientos distintos
Subclase Responsibility	Convención Smalltalk que lanza error si la subclase no redefine el método
Tipado estructural	Compatibilidad de tipos basada en la forma (campos/métodos), no en el nombre de la clase
Tipado nominal	Compatibilidad de tipos basada en la declaración explícita (implements, extends)
abstract	Keyword TypeScript que marca clase o método como no instanciable/implementable solo en subclasses
extends	Keyword para herencia de clase
implements	Keyword para declarar que una clase cumple una interfaz

Término	Definición
private	Modificador de acceso — solo accesible dentro de la clase
protected	Modificador de acceso — accesible en la clase y sus subclases
readonly	Campo inmutable después de la construcción
super()	Llamada al constructor del padre — obligatoria en subclases con constructor

7. Referencias

Bibliografía del curso (peer-reviewed)

- **Gabbrielli, M. & Martini, S. (2023).** *Programming Languages: Principles and Paradigms* (2nd ed.), Springer, Cap. 10: “Object-Oriented Paradigm”. [Fuente principal del contraste Simula → Smalltalk → OO moderno]
- **Sebesta, R.W. (2019).** *Concepts of Programming Languages* (12th ed.), Pearson, Cap. 12: “Support for Object-Oriented Programming”. [Referencia estándar sobre diseño de lenguajes OO]
- **Sweller, J., Chen, O., et al. (2023).** *Extending Cognitive Load Theory to incorporate Working Memory Resource Depletion*, Educational Psychology Review, 35. [Fundamento pedagógico del diseño de la guía]

Para profundizar en Smalltalk

- **Goldberg, A. & Robson, D. (1983).** *Smalltalk-80: The Language and its Implementation*, Addison-Wesley. (fundacional — lectura histórica)
- **Ducasse, S. et al. (2023).** *Pharo by Example 90* (libre online): <https://books.pharo.org/pharo-by-example90/>

Para profundizar en TypeScript

- **Documentación oficial TypeScript** — Clases: <https://www.typescriptlang.org/docs/handbook/>
- **TypeScript Playground** (editor en línea): <https://www.typescriptlang.org/play>

Entorno de práctica

- **Pharo Playground** (Smalltalk moderno): <https://pharo.org/>
- **TypeScript Playground**: <https://www.typescriptlang.org/play>
- **SWISH** (SWI-Prolog online — para comparar con T06/T07): <https://swish.swi-prolog.org/>